Clay Ramey

COMP 5710

Workshop 3

February 25, 2026

Flow of Execution for calc.py

Overview

This document goes over the flow of execution for calc.py, which is basically a simple calculator program. The main goal here is to track how a tainted data value (1000) moves through the program using taint tracking.

Program Structure

calc.py has two main parts:

1. A function called simpleCalculator(v1, v2, operation) that does the math

2. The main block (if __name__=='__main__':) which is where the program starts running

Detailed Execution Flow

Step 1: Initial Variable Assignment (Line 24)

```
input1 = 1000
```

So here 1000 gets assigned to input1. This is our taint source, basically the value we want to follow through the whole program.

Data flow: 1000 -> input1

Step 2: Tuple Assignment (Line 25)

```
val1, val2, op = input1, 5, '*'
```

This is a multiple assignment thing (tuple unpacking). val1 gets the value from input1 which is 1000, val2 gets 5, and op gets the string '*'. This just sets up the arguments for the function call.

Data flow: input1 -> val1

Step 3: Function Call (Line 26)

```
data = simpleCalculator(val1, val2, op)
```

Now it calls the simpleCalculator function. val1 (1000) goes to parameter v1, val2 (5) goes to v2, and op ('*') goes to operation.

Data flow: val1 -> v1

Step 4: Function Execution - Initial Assignment (Line 8)

res = 0

Inside the function, res gets set to 0 first. Its just the default before anything happens.

Step 5: Conditional Logic (Lines 9-18)

```
if operation=='+': res = v1 + v2
elif operation=='-': res = v1 - v2
elif operation=='*': res = v1 * v2
elif operation=='/': res = v1 / v2
elif operation=='%': res = v1 % v2
```

The function checks what operation is. Since operation is '*', it goes to the multiplication line. So res = 1000 * 5 = 5000.

Data flow: v1 -> res (the tainted value 1000 flows into the result)

Step 6: Function Return (Line 19)

```
return res
```

The result 5000 gets returned back to the caller and stored in data.

Data flow: res -> return value -> data

Step 7: Result Display (Line 27)

```
print('Value#1:{} \nValue#2:{} \nOperation:{} \nResult:{}'.format(val1, val2, op, data))
```

This prints out:

Value#1: 1000
Value#2: 5
Operation: *
Result: 5000

Taint Tracking Analysis

Taint Flow Path

The full path of the tainted value (1000) through the program:

1000 -> input1 -> val1 -> v1 -> res

Explanation of Each Step

1. 1000 - the starting taint source (just a number)

2. input1 - first variable it gets put into

3. val1 - second variable through the tuple unpacking

4. v1 - the function parameter that gets the tainted value

5. res - the result variable thats computed using the tainted value

How the Taint Spreads

1. Direct assignment: input1 = 1000

2. Tuple assignment: val1, val2, op = input1, 5, '*'

3. Function call: simpleCalculator(val1, ...)

4. Parameter binding: val1 connects to v1 when the function starts

5. Binary operation: res = v1 * v2 - tainted value used in math

Abstract Syntax Tree (AST) Analysis

The program uses pythons ast module to pull out the program structure. Here are the dataframes it builds:

Variable Assignments (VAR_DF)

LHS	RHS	TYPE
input1	1000	VAR_ASSIGNMENT
val1	input1	VAR_ASSIGNMENT
val2	5	VAR_ASSIGNMENT
op	*	VAR_ASSIGNMENT

Function Calls (CALL_DF)

LHS	FUNC_NAME	ARG_NAME	TYPE
data	simpleCalculator	val1	FUNC_CALL_ARG:1
data	simpleCalculator	val2	FUNC_CALL_ARG:2
data	simpleCalculator	op	FUNC_CALL_ARG:3

Function Definitions (FUNC_DEF_DF)

FUNC_NAME	ARG_NAME	TYPE
simpleCalculator	v1	FUNC_DEFI:1
simpleCalculator	v2	FUNC_DEFI:2
simpleCalculator	operation	FUNC_DEFI:3

Function Variables (FUNC_VAR_DF)

LHS	RHS	TYPE
res	0	FUNC_SINGLE_ASSIGNMENT
res	,v1,v2	FUNC_VAR_ASSIGNMENT

Implementation Details

Tracking Algorithm

The trackTaint() function works like this:

1. Start with the taint value (1000)
2. Look in VAR_DF for the first assignment (input1 = 1000)
3. Look in VAR_DF again for the next one (val1 = input1)
4. Check CALL_DF to find where val1 gets used in a function call
5. Check FUNC_DEF_DF to map the argument to the parameter (val1 -> v1)
6. Check FUNC_VAR_DF to find where v1 is used (res = v1 * v2)
7. Put the path together by joining all the tracked variables with ->

Pandas Operations Used

- Boolean indexing: df[df['column'] == value]
- Multiple conditions: df[(df['col1'] == val1) & (df['col2'] == val2)]
- String contains: df['col'].str.contains(pattern)
- Type conversion: .astype(str) for string stuff

Security Implications

This taint tracking stuff is useful for:

1. Input validation - tracking user inputs through the program
2. Finding vulnerabilities - seeing where untrusted data gets to sensitive operations
3. Information flow control - making sure sensitive data doesnt leak
4. Bug detection - finding data dependencies you didnt expect

In this example, if 1000 was user input, we can check that it goes through the right steps before being used in calculations.

Conclusion

This shows a pretty simple but complete example of data flow tracking. The tainted value (1000) goes from initial assignment to variable storage to function argument to function parameter to the computation result.

The automated taint tracking works by using ast parsing and dataframe queries, and it gives us the expected output: 1000->input1->val1->v1->res

References

- Python AST module docs: https://docs.python.org/3/library/ast.html
- Pandas DataFrame selection: https://pandas.pydata.org/docs/user_guide/10min.html#selection
- Workshop 3 repo: https://github.com/effat/SQA-2026/tree/main/Workshops/Workshop-3